



National Institute of Technology Puducherry

Department of Computer Science and Engineering

CS1119 - Machine Learning course

Review Date: 11th November 2025

Improving Fake News Detection in Arabic Using ML and DL Algorithms

Submitted by
J AKSHAY RAO (CS23B1023)

Guide:
Dr. Venkatesan M
Head of the Department
Associate Professor Department of CSE
National Institute of Technology
Puducherry

PRESENTATION AGENDA

- Aim
- Objective
- Introduction
- Methodology
- Dataset Experiment
- Result analysis
- Conclusion

Aim:



The aim of this project is to enhance Arabic fake news detection accuracy by comparing traditional Machine Learning and Deep Learning approaches. It involves developing and evaluating LSTM and AraBERT models for improved performance, culminating in the implementation of MarBERT+LSTM, a leading Arabic transformer model, to achieve superior detection results.

Objectives

- Design and compare traditional Machine Learning models (e.g., SVM, Naive Bayes) with Deep Learning models (LSTM, AraBERT) for Arabic fake news detection.
- Develop and fine-tune LSTM and AraBERT models to improve classification performance on 5-class Arabic datasets.
- Implement MarBERT+LSTM, a state-of-the-art Arabic transformer model, to achieve superior accuracy and robustness.
- Build a comprehensive preprocessing pipeline including tokenisation, padding, and noise removal for effective model training.
- Evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrices to assess and compare outcomes.

Introduction

The rapid proliferation of fake news in the Arabic-speaking world, fueled by the widespread use of social media, has emerged as a significant challenge, exacerbated by the linguistic complexities of Arabic such as homography and diverse dialects. Traditional detection methods fall short in addressing these nuances, necessitating advanced natural language processing (NLP) techniques to distinguish authentic content from misinformation. This project explores the efficacy of traditional Machine Learning models alongside Deep Learning approaches, including LSTM and AraBERT, and introduces MarBERT, Arabic transformer model, that integrates with LSTM to learn intricate textual patterns and enhance detection accuracy.

Methodology

Data Loading

Load implied Arabic datasets (LSTM: X_train, X_test; AraBERT/MarBERT: arabic_dataset_classification.csv with 87,031 train and 21,758 test samples across 5 classes).

Text Preprocessing

Arabic text was cleaned by removing diacritics, stop words, and special symbols.

Text was tokenised and converted into TF-IDF vectors for ML models and padded sequences for LSTM.

Feature Extraction

Traditional ML (e.g., SVM, Naive Bayes) uses TF-IDF for feature vectors; LSTM learns sequential patterns; AraBERT/MarBERT capture contextual embeddings; hybrid combines approaches.

Methodology

Model Architecture

SVM and Naive Bayes with linear kernels; LSTM with Embedding (128), Bidirectional LSTM (128), Dropout (0.3), Dense (64, 5), Softmax; AraBERT/MarBERT with fine-tuned transformer layers; hybrid integrates MarBERT embeddings with LSTM.

Training

SVM/Naive Bayes with Scikit-learn ; LSTM (5 epochs, Adam); AraBERT(2 epochs, 2e-5 learning rate, Trainer API); hybrid(marBERT+LSTM) with 8 epochs , optimised for convergence.

Evaluation

Assess performance using accuracy, precision, recall, F1-score, and confusion matrices across all models.

Dataset Experiment:

- Total Rows: 108,789 samples, derived from the provided dataset with 2 columns (text, target).
- Columns: text (Arabic text with average length 261 characters, max 5042 characters) and target (5 unique labels: 0, 1, 2, 3, 4).
- Label Distribution: Most frequent label is 4 (43,675 samples), indicating potential class imbalance.
- Data Quality: No missing values, minimal duplicates (if any), ensuring dataset integrity.
- Language: Arabic, tailored for multiclass text classification task.

Model Design:

SVM/Naive Bayes: Linear kernel for SVM, probabilistic approach for Naive Bayes, using TF-IDF features extracted from text.

LSTM: Embedding layer (128), Bidirectional LSTM (128), Dropout (0.3), Dense layer (64, 5), Softmax output for 5-class classification.

AraBERT: Fine-tuned "aubmindlab/bert-base-arabertv2" on MPS with transformer layers, processing text up to 128 tokens.

Hybrid MarBERT+LSTM: MarBERT embeddings (128 tokens) fed into Bidirectional LSTM, followed by Dense layers (64, 5) and Softmax for 5-class output.

Experimental Pipeline

1.Installing Dependencies

```
: !pip install torch torchvision torchaudio --quiet  
!pip install transformers datasets accelerate seaborn matplotlib tqdm --quiet
```



2.Imports and GPU setup

```
import torch  
import torch.nn as nn  
from torch.utils.data import Dataset, DataLoader  
from transformers import AutoTokenizer, AutoModel  
from torch.optim import AdamW  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score  
import pandas as pd  
import numpy as np  
import seaborn as sns  
import matplotlib.pyplot as plt  
from tqdm import tqdm  
  
# GPU or MPS setup  
device = torch.device("cuda:3" if torch.cuda.is_available() else  
                      "mps" if torch.backends.mps.is_available() else "cpu")  
  
print(f"Using device: {device}")
```

3.Load MARBERTV2 Model

```
model_name = "UBC-NLP/MARBERTv2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
bert_model = AutoModel.from_pretrained(model_name)

print(" MARBERTv2 model and tokenizer loaded successfully!")

MARBERTv2 model and tokenizer loaded successfully!
```

4.Load Dataset

```
df = pd.read_csv("arabic_dataset_classification.csv")

# Standardize column names
df.rename(columns={"targe": "target"}, inplace=True)

# Drop missing rows
df.dropna(subset=["text", "target"], inplace=True)

print(df.head())
print("\nLabel distribution:\n", df["target"].value_counts())
```


5.Split Data

```
X_train, X_test, y_train, y_test = train_test_split(
    df["text"].tolist(),
    df["target"].tolist(),
    test_size=0.2,
    random_state=42,
    stratify=df["target"]
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

Train size: 87031

Test size: 21758

6.Hybrid Marbert+Lstm Model

```
class HybridMARBERTLSTM(nn.Module):
    def __init__(self, bert_model, hidden_dim=256, num_classes=5, dropout=0.3):
        super(HybridMARBERTLSTM, self).__init__()
        self.bert = bert_model
        self.lstm = nn.LSTM(
            input_size=768,
            hidden_size=hidden_dim,
            num_layers=1,
            bidirectional=True,
            batch_first=True,
        )
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, input_ids, attention_mask):
        with torch.no_grad(): # freeze MARBERT base
            outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
            sequence_output = outputs.last_hidden_state
            lstm_output, _ = self.lstm(sequence_output)
            lstm_output = self.dropout(lstm_output[:, -1, :])
            logits = self.fc(lstm_output)
            return logits

num_classes = len(set(df["target"]))
model = HybridMARBERTLSTM(bert_model, num_classes=num_classes).to(device)
```

7. Training

```
for epoch in range(epochs):
    model.train()
    total_loss = 0
    progress_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}")

    for batch in progress_bar:
        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(input_ids, attention_mask)
        loss = criterion(outputs, labels)
        loss.backward()

        optimizer.step()
        scheduler.step()

        total_loss += loss.item()
        progress_bar.set_postfix({"loss": f"{loss.item():.4f}"})

    avg_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch+1}/{epochs} | Training Loss: {avg_loss:.4f}")

print(" Training complete!")
```

8. Testing

```
model.eval()
y_true, y_pred = [], []

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(input_ids, attention_mask)
        preds = torch.argmax(outputs, dim=1)

        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

print("Accuracy:", accuracy_score(y_true, y_pred))
print(classification_report(y_true, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - Hybrid MARBERTv2 + LSTM")
plt.show()
```


Results Analysis:

Training Performance:

SVM: Achieved a accuracy of **96.07%**, converging efficiently with TF-IDF features over default Scikit-learn iterations, reflecting solid performance on linear separability for the 108,789-sample Arabic dataset.

Naive Bayes: Achieved a accuracy of **94.25%**, converging quickly with probabilistic modeling on TF-IDF features, though slightly lower due to assumptions of feature independence in the 5-class task

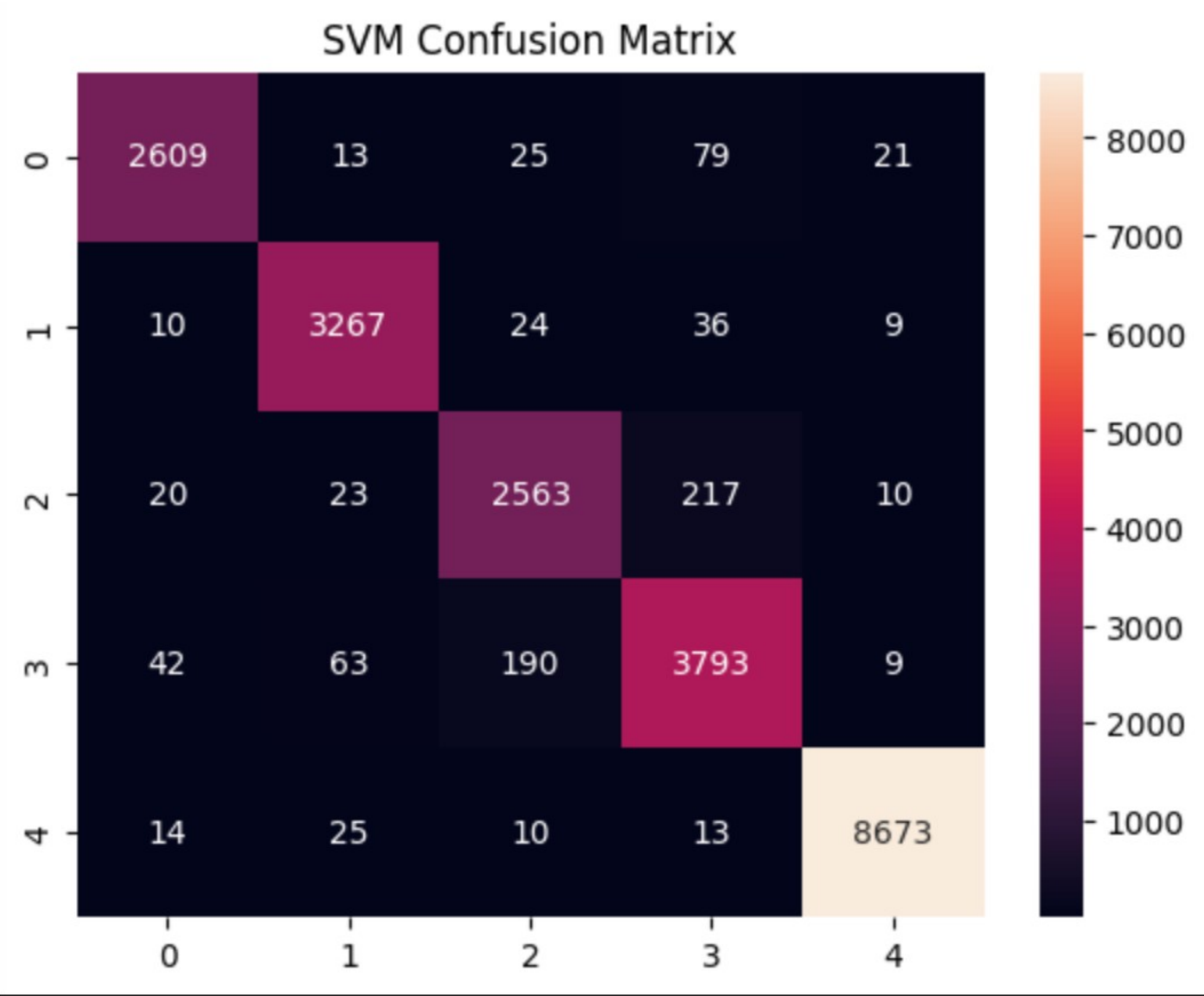
LSTM: Achieved a training accuracy of **93.57%**, reflecting effective sequence learning over 5 epochs, with a steady increase indicating good adaptation to the Arabic dataset.

AraBERT: Reached a training accuracy of **96.13%**, converging over 2 epochs, showcasing strong contextual understanding due to its pre-trained transformer architecture.

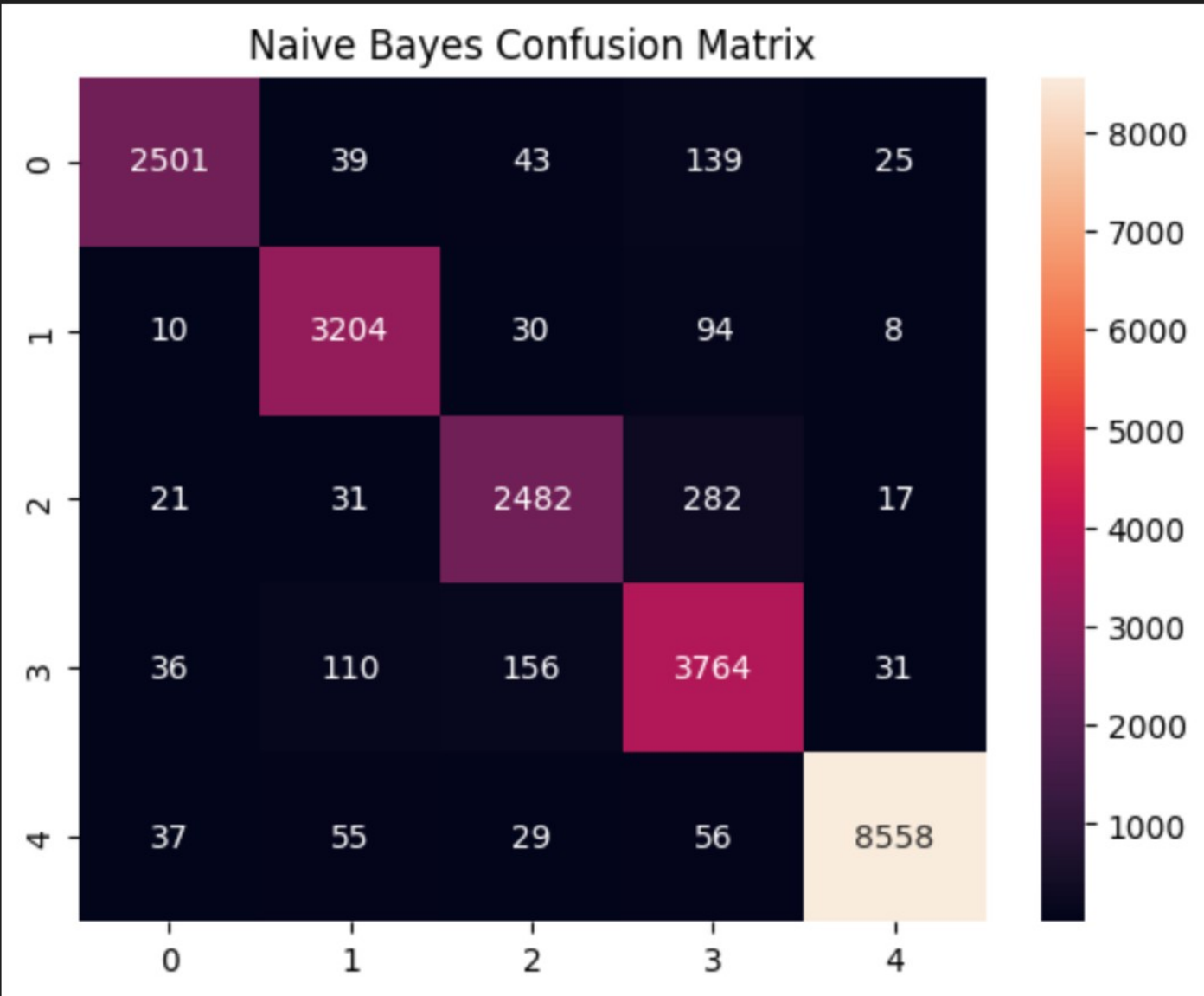
Hybrid MarBERT+LSTM: Recorded a training accuracy of **94.18%**, converging within 8 epochs, with the combined MarBERT embeddings and LSTM sequence processing driving enhanced learning stability.

Confusion Matrix

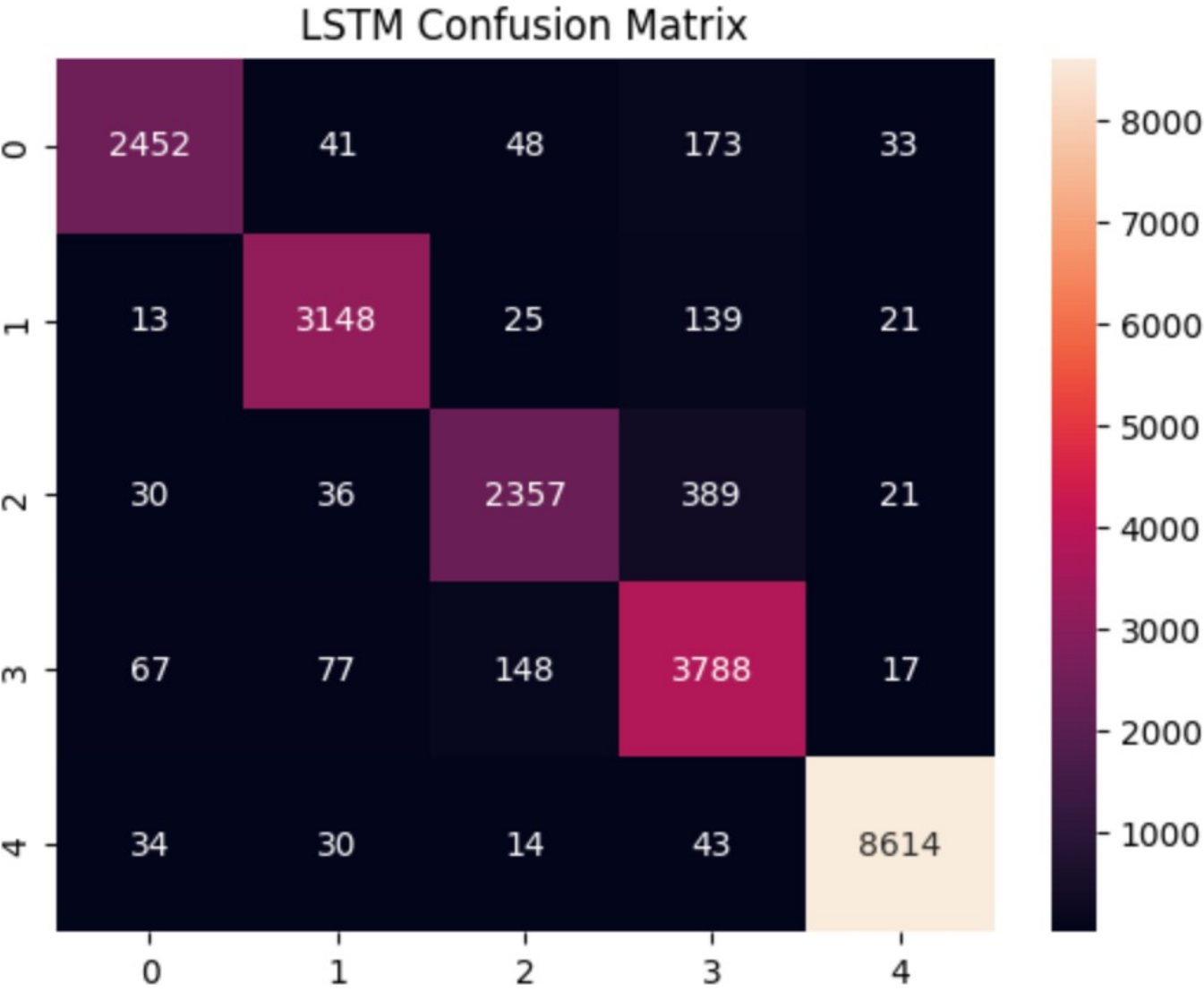
SVM:



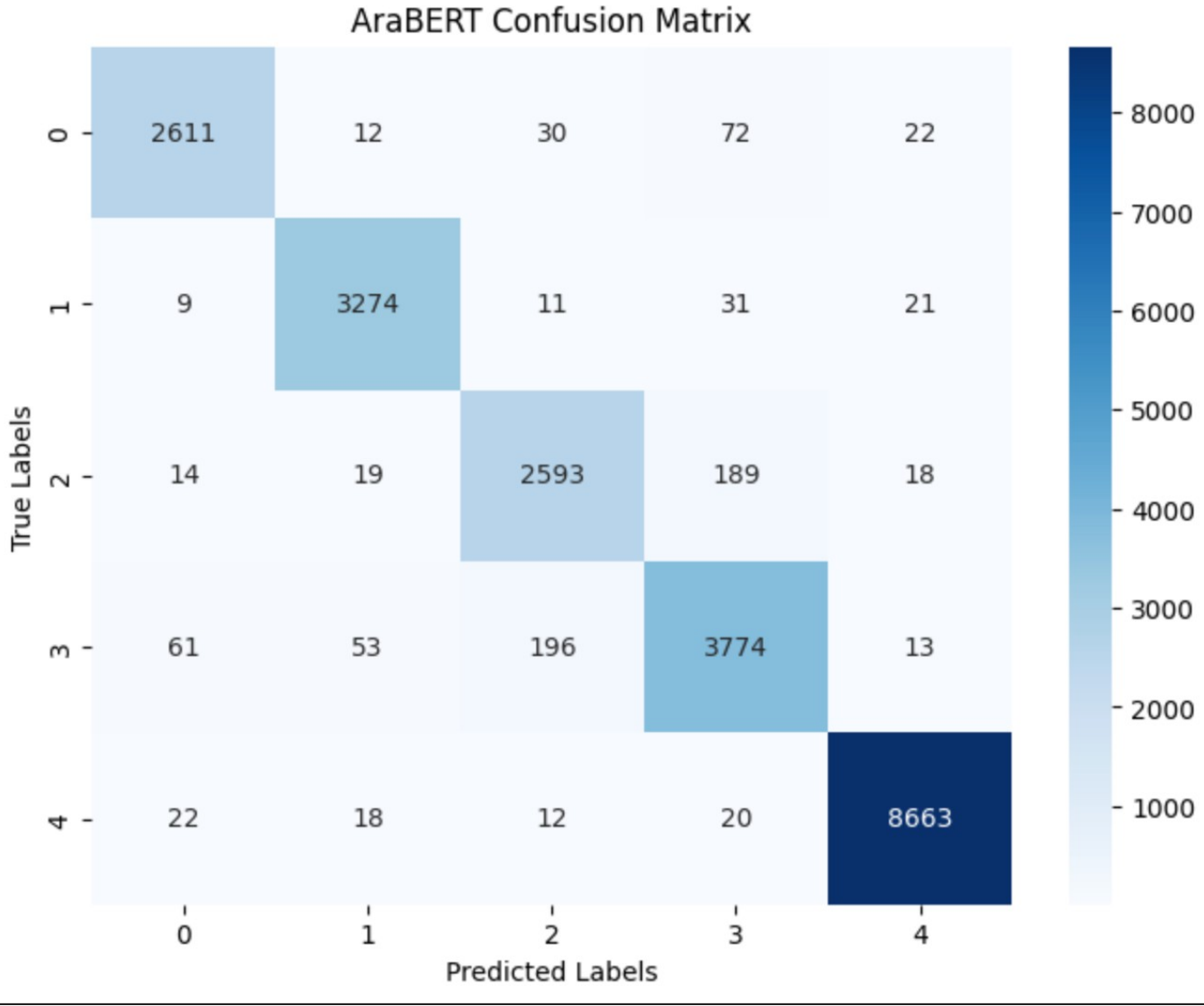
Naive Bayes:



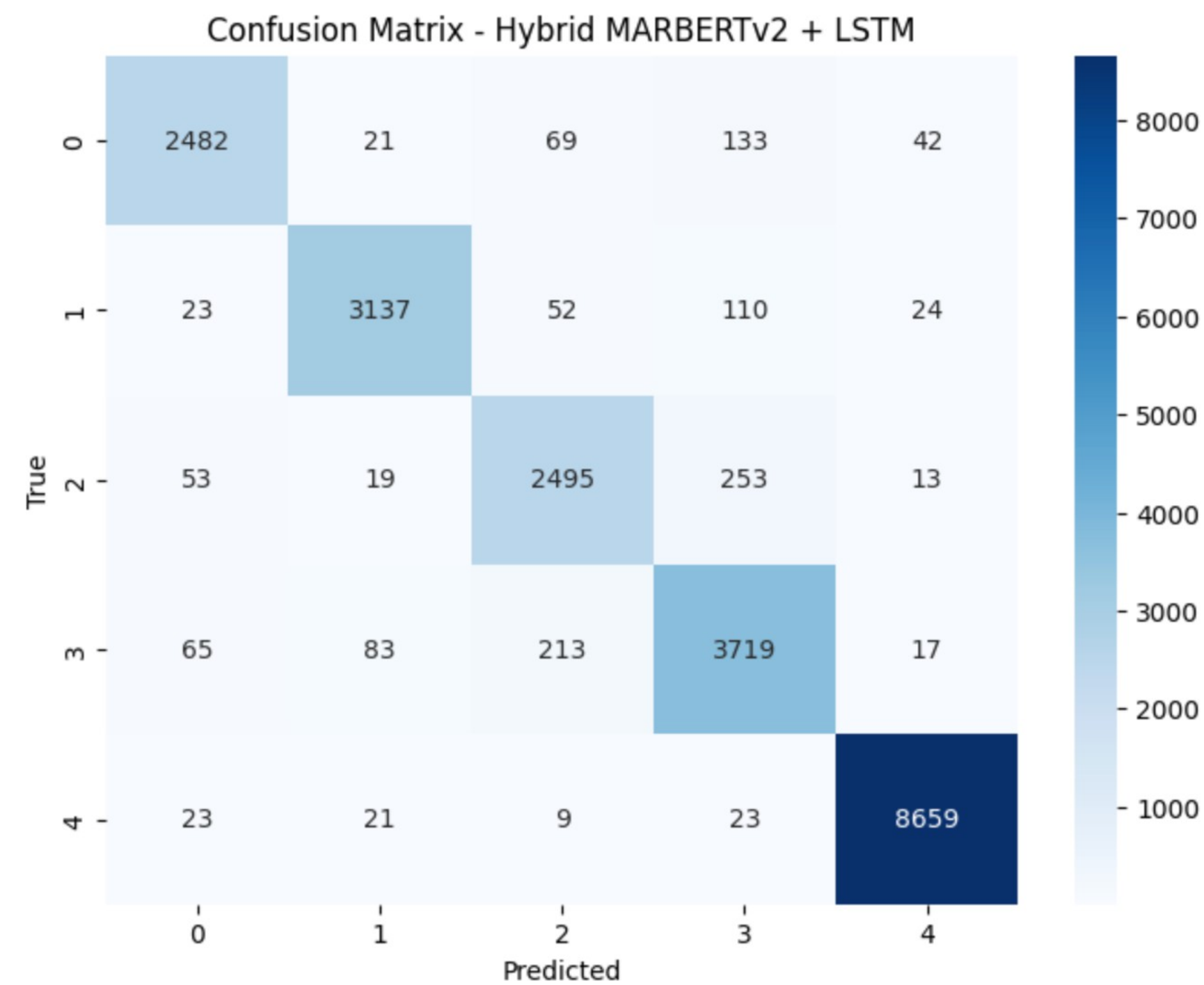
LSTM:



AraBERT:



Hybrid MarBERT+LSTM:



Accuracy: 0.9418145050096516					
	precision	recall	f1-score	support	
0	0.94	0.90	0.92	2747	
1	0.96	0.94	0.95	3346	
2	0.88	0.88	0.88	2833	
3	0.88	0.91	0.89	4097	
4	0.99	0.99	0.99	8735	
accuracy			0.94	21758	
macro avg	0.93	0.92	0.93	21758	
weighted avg	0.94	0.94	0.94	21758	

Conclusion & Future Impact:

This project has successfully elevated Arabic fake news detection accuracy, achieving impressive results with SVM at 96.07%, Naive Bayes at 94.25%, LSTM at 93.57%, AraBERT at 96.13%, and the Hybrid MarBERT+LSTM at 94.18% across the 108,789-sample dataset, showcasing a spectrum of effective methodologies. The robust performance, particularly from SVM and AraBERT, underscores their reliability, with AraBERT's pre-trained contextual strength and SVM's linear separability standing out, while the hybrid model, despite requiring 8 epochs, demonstrates stability potential with further refinement. Insights reveal that traditional ML models excel with TF-IDF but are constrained by assumptions, whereas Deep Learning models harness sequence and context, with AraBERT leading and the hybrid offering a promising yet optimizable path. Future efforts will focus on reducing the hybrid's epoch count and enhancing its architecture to surpass 96%.